

ThreadTrack

Case Study

Predicting delivery delay and return risk for D2C menswear supply chains

Anusha Chakraborty

Industrial Engineering and Supply Chain · National Institute of Fashion Technology
anushachakraborty26@gmail.com

A six-phase applied data science project — from industry research to deployed decision tools.
May 2026

1. Executive summary

Indian direct-to-consumer (D2C) menswear moves fast. Brands like Snitch drop more than a hundred new styles a week, and at that speed a supply chain slip is invisible until a customer complains — by which point it is already a refund, a return, or a lost repeat purchase. Roughly one in five online fashion orders in India is sent back, and delayed orders are returned more often still. Most brands manage both problems reactively: they discover that an order was a problem only after it already is one.

ThreadTrack asks a different question. On the day a purchase order is placed — before any fabric is cut — can we predict whether it will be delivered late or sent back, and explain why, in terms a merchandiser can act on? It answers that question with a two-tier risk model — a transparent rule-based scorer and a machine learning model — validates the answer against the unfiltered voice of real customers, and delivers it as two working tools.

The results are deliberately honest. Delivery delay predicts strongly: the machine learning model scores an AUC of 0.847. Return risk does not — it scores 0.571, barely above guesswork — and establishing that, rigorously, is the single most valuable result in the project. It is not a broken model; it is a true fact about the problem, confirmed three independent ways inside the modelling and then explained, in their own words, by 1,189 real customer reviews. A delay is caused by factors visible on the day of the order; a return is caused by what happens after the parcel arrives.

ThreadTrack was built over six phases, from industry research to deployed tools. It is designed to demonstrate one complete, honest piece of applied data science: sourcing and judging research, generating and calibrating a dataset, building and evaluating both rule-based and machine learning models, applying a large language model for genuine independent validation, and turning all of it into something a real person can use.

Result	Detail
Synthetic dataset	5,000 purchase orders, calibrated to cited industry benchmarks — 45.6% delayed, 32.9% returned (v2 dataset with upstream features)
Delay prediction	XGBoost AUC 0.847 — strong; delay is predictable from order-time data
Return prediction	XGBoost AUC 0.571 — weak, and that is the finding
Rule scorer separation	Delay risk bands separate the real delay rate 15% / 55% / 95%; return bands only 24% / 29% / 45%
Independent validation	1,189 real Snitch customer reviews classified into six supply chain themes via the Claude API
Delivered as	An Excel risk workbook, a four-tab interactive dashboard, a ranked action playbook, and a callable rule + ML hybrid

2. The problem: a supply chain that fails quietly

Every clothing order travels the same path. A purchase order is placed with a vendor; fabric is sourced; the garment is cut, sewn and finished; and the finished goods are shipped to a distribution centre and then to the customer. Two things can go wrong at the end of that path — the order can arrive late, or the customer can send it back. ThreadTrack is built around those two failure modes, delay and return, because together they account for most of the avoidable financial damage in a D2C fashion supply chain.

Why delays cost money

A delay has visible costs — refunds, expedited shipping to catch up, and staff time spent on complaints. Its larger costs are invisible: the customer who cancels, the customer who never orders again, and the public one-star review that quietly discourages dozens of future buyers. A brand that cannot anticipate its delays absorbs all of this reactively, after the damage is already done.

Why returns are tied to delays

Returns are an outsized problem for online fashion, because a customer cannot touch the fabric or try the garment on before buying. And returns are not independent of delays: a delayed order is returned more often than an on-time one. The reasons are human — the occasion has passed, the customer has moved on, or, tired of waiting, they have bought a replacement elsewhere. A delay therefore does not simply cause a delay; it quietly raises the chance of a return. A system that predicts only one of the two misses part of the cost.

Why prediction beats reaction

Most delay tools are built without supply chain domain knowledge, and they miss what an experienced buying merchandiser knows intuitively: that fabric category interacts with season, that a new vendor slips more in its first production cycle, and that deliveries to tier-2 and tier-3 cities carry risk a vendor's headline service level never surfaces. Prediction at the moment of ordering changes the order of events. If a brand can estimate, on the day a PO is placed, that it is likely to run late, it can act before anything has gone wrong — move the order to a more reliable vendor, build in a lead-time buffer, hold a small safety stock, or simply warn the customer early and honestly. Prediction converts firefighting into planning. That conversion is the entire purpose of ThreadTrack.

3. What I built: the system end to end

ThreadTrack is a pipeline — a sequence of stages where the output of each feeds the next. It was built in six phases, and is best understood as two independent tracks that converge at a single validation point.

The **synthetic track** is the prediction engine. Cited industry research is distilled into a benchmark document; those benchmarks are used to generate a realistic 5,000-order dataset; a rule-based scorer and a machine learning model each score every order; and the two are blended into a hybrid. The **real-data track** is the check on that engine: real customer reviews are collected and classified, by a large language model, into supply chain complaint themes. The two tracks meet at validation, where the model's account of risk is tested against what real customers independently say. A final output layer turns the working system into usable tools.

Phase	What it produced
1 — Foundation	Development environment, code repository, and a researched benchmark document
2 — Synthetic data	A 5,000-order dataset calibrated to industry benchmarks, with exploratory analysis
3 — Rule-based scorer	A transparent 15-factor delay and return risk scorer
4 — Machine learning	Trained XGBoost delay and return models, blended into a performance-weighted hybrid
5 — Language-model layer	1,189 real reviews classified into supply chain themes; independent model validation
6 — Output layer	An Excel risk workbook and an interactive dashboard (case study and deployment in progress)

4. The key decision: building the data honestly

A prediction model learns from examples. ThreadTrack needed a large table of past purchase orders, each labelled with whether it was eventually delayed or returned. No clothing brand publishes that kind of private operational data. So I generated it.

This invites an obvious and serious objection: if the data is invented, is the model not simply learning patterns I invented? I took that objection seriously from the start, and the answer is calibration. The synthetic data is not invented freely. Every statistical setting that shapes it — how often orders run late, how return rates vary by payment method, how long production takes by fabric, how much a manufacturing cluster's capacity crunch amplifies lead time — is drawn from a cited industry figure. Sources span a global fashion-industry report, the Ministry of Textiles annual report, and a specialised national apparel study. The data is artificial in origin but realistic in shape.

Calibration is a loop, not a setting

The first calibration pass on the v1 dataset reported 77% of orders delayed — far higher than any real business would survive. The cause was diagnosed: several risk factors were being multiplied together, and multiplication compounds fast. The settings were softened; the second version read 57%, still too high; a single seasonal effect was over-applied and corrected. The third version read 31.7% delayed and 29.4% returned — inside the benchmark range. A later v2 iteration that added five upstream operational features ran the same three-pass loop independently — 80% → 52% → 45.6% — producing the dataset this case study now references. Three passes — twice — is not sloppiness. It is the normal rhythm of modelling: build, measure, diagnose, correct, repeat. What matters is that every correction was driven by a specific diagnosed cause, not a guess.

The discipline that made this safe is a strict separation of parameters from logic. Every benchmark number lives in one configuration file; the generation logic lives in another and is never touched when a number changes. When a figure is corrected — or when expert validation arrives — only the parameter file is edited, and the logic cannot be accidentally broken.

5. The model: two methods with opposite strengths

ThreadTrack does not rely on a single prediction method. It uses two, on purpose, because they fail in opposite ways.

The rule-based scorer is a system of fifteen hand-weighted supply chain factors — vendor reliability, fabric category interacting with season, destination tier, payment mode, order quantity, a heavy penalty for an unproven first-cycle vendor, and more. Each factor adds a fixed number of points to an order's delay score and its return score, both on a 0-to-100 scale. Its strength is total transparency: every score can be explained, line by line, in language a merchandiser already uses. Its weakness is that it can only contain knowledge the designer already has — it cannot discover anything new. Designing this rubric, and justifying every weight, is the genuine domain-knowledge core of the project.

The machine learning model is an XGBoost classifier trained on the 5,000-order dataset. Its strength is the mirror image of the rule scorer's: it can find subtle combinations of factors that no human would think to write down. Its weakness is that it is a black box — it is hard to explain why it made any single prediction. It was trained and tested honestly, with a fifth of the data held back and unseen, so its measured performance is real and not memorised.

The two were then blended into a hybrid, and that step taught its own lesson. The first attempt, an equal 50/50 blend, scored slightly below the better of the two methods used alone. The reason is clear in hindsight: the two are not equally good at each task, and giving the weaker one equal weight simply dilutes the stronger. The corrected hybrid weights each blend toward whichever method is stronger for that task, and then matches the best single method. But the hybrid's real value was never a higher accuracy number. It is two things a number cannot capture: it always carries the rule scorer's plain-English reasons, so every prediction can be explained to the person acting on it; and it compares the two methods on every order, agrees on roughly 84% of them, and flags the remaining 16% — where they disagree — for a human to review. That is a usable working procedure, not a number.

6. The central finding: delays are predictable, returns are not

The most important result in ThreadTrack is not its best score. It is a limit, surfaced deliberately and confirmed from every angle: delivery delay can be predicted from order-day information, and return risk largely cannot.

Three independent methods inside the modelling agree on it:

- **Rule scorer separation.** Sorted into low, medium and high delay-risk bands, orders show real delay rates of 15%, 55% and 95% — a more than sixfold spread. The same orders sorted by return-risk band show return rates of just 24%, 29% and 45% — barely a twofold spread.
- **Machine learning performance.** The delay model scores an AUC of 0.847, comfortably strong. The return model scores 0.571, barely above the 0.5 of pure chance.
- **Feature-importance shape.** The delay model concentrates its attention — a single factor accounts for more than a third of its decisions. The return model spreads its attention thinly across many factors, with several meaningless details ranking high. Concentrated importance is the fingerprint of a model that found real structure; smeared importance is the fingerprint of one that found very little.

These are not three versions of the same test. A hand-built rubric, a learned algorithm, and a diagnostic of that algorithm's internals are different instruments. When all three agree, the finding is real.

Why a near-failure is the project's best result

It would have been easy to hide the weak return number, or to over-tune the model until it looked respectable. ThreadTrack does the opposite, because the weak result is true and useful. A delay is caused by factors that exist on the day an order is placed — the vendor, the cluster's capacity, the fabric's lead time, the season. A return is caused by factors that do not yet exist at that moment — whether the garment will fit, whether its quality will meet expectations, and whether it will end up late. No system can predict, from order-day data, something that is partly determined by later and partly random events. Stating that limit precisely is better analysis than hiding it.

The practical consequence is clear. The delay score is reliable enough to plan against. The return score should be read as a weak directional signal, not a precise prediction — and a brand serious about returns must look to post-purchase data, especially sizing and quality feedback. That is exactly where the next phase went.

7. Validation: checking the model against real customers

Every number in the synthetic dataset was shaped by my own research. That makes it vulnerable to one fair criticism — circularity: a model trained on assumption-shaped data may simply be learning those assumptions back. Phase 5 is the answer to that criticism, and it required a source of evidence the project did not create.

Customer reviews are that source. The reference brand, Snitch, has a mobile app with a public page of customer reviews. A short program collected about 1,200 of them, each with its star rating and written text. These reviews are real and — crucially — independent: they were written by people with no knowledge of this project and no connection to its assumptions.

Raw review text cannot be analysed directly, so a large language model — the Anthropic Claude API — read each review and classified it into a fixed schema of six supply chain themes: a delivery delay, a return or refund problem, a sizing or fit problem, a product-quality problem, a customer-service problem, or none of these. The schema was multi-label, because a real complaint often is more than one thing at once. The model was instructed to judge only the words and never the star rating, and its replies were locked to a strict machine-readable shape using structured outputs. The pipeline ran in batches, was built to resume safely if interrupted, and classified 1,189 reviews for roughly one US dollar of API cost.

The comparison that followed is a triangulation. A synthetic order and a real customer cannot be matched one to one, so this is not a row-by-row check. It is a test of whether two entirely independent sources — a benchmark-calibrated model and the unfiltered voice of 1,189 customers — tell the same story. Four things emerged:

- **Scope confirmed.** Delivery delays and return problems, the two outcomes the model predicts, are both among the most common complaint themes. The model is built around problems customers actually have.
- **Returns dominate.** Return and refund complaints outnumbered delivery-delay complaints by roughly two to one — confirming that returns are a central concern, worth modelling even though they are the harder of the two to predict.
- **The central finding, explained.** Sizing complaints and product-quality complaints were present and specific. Those are precisely the post-purchase factors that order-day data cannot contain — the reviews name, in customers' own words, exactly why the return model is weak.
- **An honest gap.** The single most common theme was poor customer service, which the model does not predict. But about three in four of those reviews also mentioned a delay or a return. Poor service is, for the most part, not a separate failure — it is the downstream symptom of the operational failures the model already predicts.

Independent corroboration is, by definition, the opposite of circular reasoning. When evidence the project did not create points at the same problems the model emphasises, the agreement cannot be an echo of the project's own design. Providing that corroboration was the entire purpose of the

phase.

8. The output layer: two tools for two users

A working system is not the same thing as a usable tool. For most of the project, the scorer and the models lived in code, runnable only by their author. Phase 6 turned the system into something a real person could pick up — and it produced two distinct tools, on purpose, because the people who would use ThreadTrack do not all work the same way.

An Excel risk workbook. A supply chain analyst, in practice, lives inside spreadsheets. So the rule scorer was rebuilt entirely in Excel formulas — not a screenshot or an export, but the scorer itself, in the one tool the analyst already trusts. An analyst types an order into a row and the workbook returns its scores, risk bands and written reasons, across four sheets, with risk colour-coded and inputs validated against fixed lists.

An interactive dashboard. A planning manager, or a reviewer of this project, wants something they can open in a browser with nothing to install. The dashboard is a four-tab web application: an order book of every scored order, filterable and sortable; a live form to score a new order; a portfolio view that rolls the whole book up into charts; and a plain-language explanation of the model.

Two design principles

The dashboard was shaped by two principles worth stating, because both are transferable. The first: **ask the user only for what they actually know.** A person placing an order knows the destination city — so they enter a city, and the system derives its delivery tier from a built-in table of about 160 Indian cities. They know which vendor they are using — so they pick it from a list, and its cluster, reliability and track record are looked up automatically. They know the date — so they enter it, and the season is derived from that date together with the vendor's region, because the monsoon reaches different manufacturing clusters at different times. The user is never asked to supply a tier, a reliability figure, or a season.

The second: **the tool informs a decision, it does not make it.** When a new order is scored, the dashboard shows the score first and then asks the user to confirm the order or discard it. The score exists to support a genuine go or no-go call; the human makes that call. ThreadTrack is built to be an assistant, not a replacement.

9. From portfolio to production

ThreadTrack version one is a portfolio demonstration, and it is scoped deliberately. It runs on free hosting. It is stateless — an order entered in the dashboard lives only for that browser session. It uses the synthetic dataset, not any company's real orders. It is a single shared application with no accounts. For a portfolio piece these are the right choices: zero running cost, nothing to maintain, and a system anyone can open from a link. But they are explicitly not what a brand would run, and knowing the difference is part of the engineering.

A real production deployment of ThreadTrack would require five things beyond the demo:

- **Order intake.** A brand does not type orders into a form. Production intake has two paths: a bulk upload of a purchase-order batch as a spreadsheet, and a direct integration with the brand's ERP or order-management system, so every new PO is scored automatically the moment it is created. The rule scorer is already written as a pure function — data in, scores out — so it sits behind either path unchanged.
- **A persistent database.** Orders, vendors, scores, and — most importantly — realised outcomes (was this order actually late? was it returned?) must be stored in a real database, not held in a browser session. Those stored outcomes are what makes the next point possible.
- **Accounts and multi-tenancy.** One deployment would serve many brands, and each must see only its own orders and vendors. That means authentication and strict tenant isolation — every record tagged to a company, every query enforcing it.
- **A retraining loop.** The version-one model is trained on synthetic data. Once a brand has accumulated its own real orders with their realised outcomes, the model is retrained on that genuine history. This is the point at which the model stops being calibrated-synthetic and becomes truly learned — and the same real history is what lets an unproven vendor's first-cycle penalty graduate to a measured track record.
- **Monitoring.** A live model must be watched: its predictions checked against outcomes as they arrive, an alert raised if its performance drifts, and a high-risk order routed to a planner who can act on it.

Dimension	Version 1 (this project)	Production
Hosting	Free shared hosting	Managed cloud, per brand
Data	5,000-order synthetic dataset	The brand's real purchase orders
Order intake	Manual entry in a form	Bulk upload and ERP integration
State	Session-only, nothing saved	Persistent database
Users	One shared app, no logins	Accounts, multi-tenant isolation
Model	Trained once on synthetic data	Retrained on each brand's outcomes

None of this changes the core of ThreadTrack — the scorer, the model and the validation logic are production-shaped already. What production adds is the infrastructure around that core: where the data comes from, where it is kept, who is allowed to see it, and how the model keeps learning.

10. Honest limitations

A portfolio project is judged partly on how clearly it sees its own edges. ThreadTrack has four limitations worth stating plainly.

- **The data is synthetic.** The model is validated for plausibility — its behaviour matches cited benchmarks and the independent voice of real customers — but it has not been proven against a single brand's real delay and return outcomes, because that data is not public. Production retraining is what would close this gap.
- **Return prediction is weak by nature.** As Section 6 sets out, this is a true property of the problem rather than a defect — but it does mean the return score must be used as a soft signal, not a hard prediction.
- **It rests on one reference brand and one market.** The review validation uses Snitch's public reviews, and the seasonal model uses the Indian calendar. The approach generalises; the specific calibration would need revisiting for another brand or country.
- **Some figures are estimates.** A few supply chain lead times are research-based estimates rather than directly cited numbers, and a request for expert validation of them is open. Because parameters are kept separate from logic, those figures can be updated with no change to any code.

None of these is hidden, because stating a limit precisely is itself a part of sound analysis.

11. What this project demonstrates

ThreadTrack is a portfolio project, and its purpose is to evidence a complete and honest piece of applied work. The skills it demonstrates are general and transferable, and each is tied to a concrete part of the project:

- **Research and source judgement** — selecting industry reports across four deliberate archetypes and triangulating their figures against one another.
- **Data engineering and calibration** — generating a synthetic dataset and correcting it through a disciplined build-measure-diagnose loop until its behaviour matched reality.
- **Transparent modelling** — designing a fifteen-factor scorer in which every weight encodes, and can be justified by, real supply chain domain knowledge.
- **Honest machine learning** — training and testing with held-out data, reporting performance with a standard metric, staying alert to leakage, and presenting a weak result as a finding rather than hiding it.
- **Applied language models** — using structured outputs, batching and a resumable pipeline to turn 1,189 unstructured reviews into genuine independent validation.

- **Product thinking** — building two tools for two distinct users, making deliberate interface decisions, and scoping a portfolio demo clearly against what production would require.
- **Communication** — this case study, a README, inline rubric and benchmark documents, plus tests, lint, and a Makefile that record every design decision in either prose or executable form.

One throughline runs through all of it. Many times in this project a choice arose between something that looked impressive and something that was honest and well-reasoned — equal-weight versus performance-weighted blending, hiding versus surfacing the weak return result, glossing over versus naming the synthetic-data objection. ThreadTrack consistently chose the honest option, because a reader who is paying attention is far more convinced by sound judgement than by an inflated claim.

12. v2 — the modelling-depth iteration

After v1 of this case study was first written, a senior apparel-tech practitioner read it and gave a sharp, specific critique: predicting the problem is not the same as solving it; upstream operational signals drive most of real delay risk; without a feedback loop a trained model is operationally symbolic; and the data has never seen the mess of a real factory ERP. v2 is the response, shipped as six concrete additions.

- **Upstream operational features.** Five signals planners actually watch — sampling delay, fabric mill slip, trims confirmation lag, factory NCR backlog, and buyer-change frequency — are now sampled per order in the generator, amplify actual outcomes, and feed six new rule-scorer factors (16–21). The model is no longer textbook features only; it carries the signals a real planner monitors.
- **A data-quality stress test.** *src/messy_data_stress_test.py* corrupts the dataset the way real factory ERPs do — typos, missing fields, distribution drift — and reports the cost in AUC. Rule scorer drops 0.801 to 0.709; a logistic-regression baseline drops 0.861 to 0.786 and recovers to 0.799 when retrained on the messy distribution. The recovery is the visible argument for the loop.
- **A simulated MLOps feedback loop.** *src/feedback_loop_simulation.py* trains a model on the first three months of the dataset, streams the remaining nine with escalating drift (5%, 15%, 30%), and retrains every two months. Under high drift the loop pulls clearly ahead of a frozen model. The output chart (*output/feedback_loop_simulation.png*) is the loop running, not just diagrammed.
- **A ranked action playbook.** *src/action_playbook.py* wraps the score with specific, ranked moves a planner can act on in fifteen seconds — escalate trims with the buyer, reserve fallback capacity at an alternate vendor, pre-book the next fabric lot, push prepaid at checkout. The score is the diagnosis; the playbook is the prescription.
- **A callable performance-weighted hybrid.** v1 named a hybrid that did not exist in runnable form. *src/hybrid_scorer.py* now loads the trained XGBoost models and the rule scorer, returns

both components plus a 0.6 / 0.4 (delay) and 0.5 / 0.5 (return) blend, and flags orders where the two methods disagree by more than 25 points for human review.

- **A REST API sketch.** *api/* is a FastAPI service over SQLite that exposes the rule scorer, the hybrid, and CRUD on orders + realised outcomes — six endpoints, eight passing tests. It closes the Section 9 production sketch in code: the production version swaps SQLite for Postgres and adds auth + tenancy, but the scoring core (already pure functions) is unchanged.
- **Engineering hygiene.** A *tests/* suite (34 pytest checks across the rule scorer, the season calendar, the action playbook, the corruption simulator, and the API), *ruff* as a project-wide linter, a *Makefile* consolidating the pipeline into single targets, and a *pyproject.toml* declaring tooling configuration. These were the basics v1 had skipped.

13. Closing

ThreadTrack takes a supply chain problem that brands normally meet reactively — delivery delays and returns — and turns it into something that can be anticipated on the day an order is placed, with a transparent reason attached to every prediction. It is equally clear about how far that prediction reaches: delays can be foreseen, returns largely cannot, and the project proves both rather than asserting them.

The system is built and validated. The portfolio v1 ships three deliverables — the Excel workbook, the interactive dashboard, and this case study — and the v2 iteration of Section 12 has added the modelling-depth changes a senior practitioner asked for. Putting the dashboard online for public access, and the continued refinement of its interface, are the remaining steps. The foundation, and the thinking behind it, are documented here, in the README, and in the source itself.

ThreadTrack is a portfolio project by Anusha Chakraborty. The purchase-order dataset is synthetic, with distributions calibrated to public industry benchmarks; the validation layer uses public Google Play Store reviews. No proprietary or confidential data is used.